

Updating Context Specific Constraints in coreConsultant

coreBuilder Application Note

Scott Knowlton, Synopsys Design Reuse Group

July 2000

Abstract

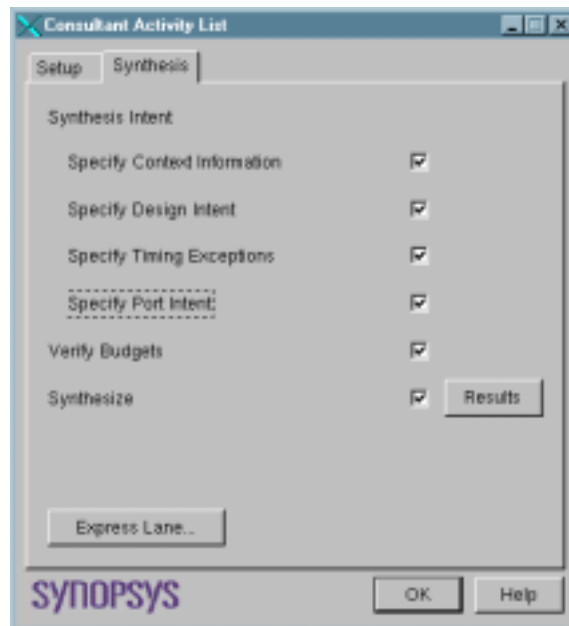
coreConsultant is used by core integrators to automatically run synthesis for cores that have been packaged with coreBuilder. This application note is primarily concerned with educating the coreConsultant user in the available methods to update context information for a core.

Introduction

One of the key features of cores packaged with coreBuilder is the default timing and loading constraints on the ports included by the core developer in the coreKit. These constraints are set by the core developer based on his knowledge of how the core is going to be used, but often may not totally reflect the context of the core in the SoC under development. In this case, the core integrator should update the timing and loading constraints on the ports to reflect the context that the core is being used. In the following sections, we will explore how to do this using the coreConsultant GUI and also through the command line.

Updating Core Context Using the GUI

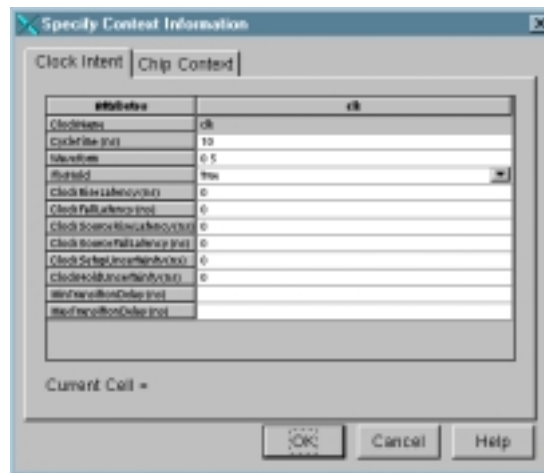
Using the coreConsultant GUI, the constraints that focus on the context of the core are primarily located in three activities on the Synthesis tab of the Activity List. The Activity list is shown in the figure below.



To update the context information, the user should first select

Updating Context Specific Constraints with coreConsultant

the *Specify Context Information* Activity. In this activity, the user specifies clock information in the spreadsheet labeled *Clock Intent*. The forms use the popular spreadsheet format, so most users will be familiar with entering this data. For our SoC, we could change the default clock cycle time by changing the “CycleTime” attribute. In the figure below, the default value set by the designer is 10ns. For our chip, we need this to be 8ns, so we just type an 8 into the form next to the “CycleTime” attribute.



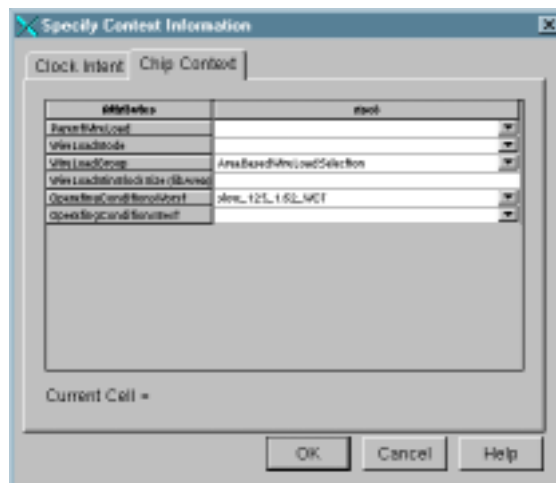
The image shows the 'Specify Context Information' dialog box with the 'Clock Intent' tab selected. The dialog has a table with two columns: 'Attribute' and 'Value'. The 'Attribute' column lists various clock-related attributes, and the 'Value' column shows their current values. The 'CycleTime' attribute is highlighted, and its value is '10'.

Attribute	Value
ClockName	clk
CycleTime (ns)	10
IsExternal	0
IsReset	True
Clock Rise Latency (ns)	0
Clock Fall Latency (ns)	0
Clock Source Rise/Fall Delay (ns)	0
Clock Source Fall Latency (ns)	0
Clock Setup Time (ns)	0
Clock Hold Time (ns)	0
Min Transition Delay (ns)	0
Max Transition Delay (ns)	0

Current Cell =

Buttons: OK, Cancel, Help

After adjusting the clock intent, click on the *Chip Context* tab and select the appropriate context info based on the users design and the technology library. For example, the user should set the attribute "OperatingConditionsWorst" to reflect the operating conditions that the user is setting for the SoC.



The image shows the 'Specify Context Information' dialog box with the 'Chip Context' tab selected. The dialog has a table with two columns: 'Attribute' and 'Value'. The 'Attribute' column lists various chip context attributes, and the 'Value' column shows their current values. The 'OperatingConditionsWorst' attribute is highlighted, and its value is 'slow_125C_1.62_MCT'.

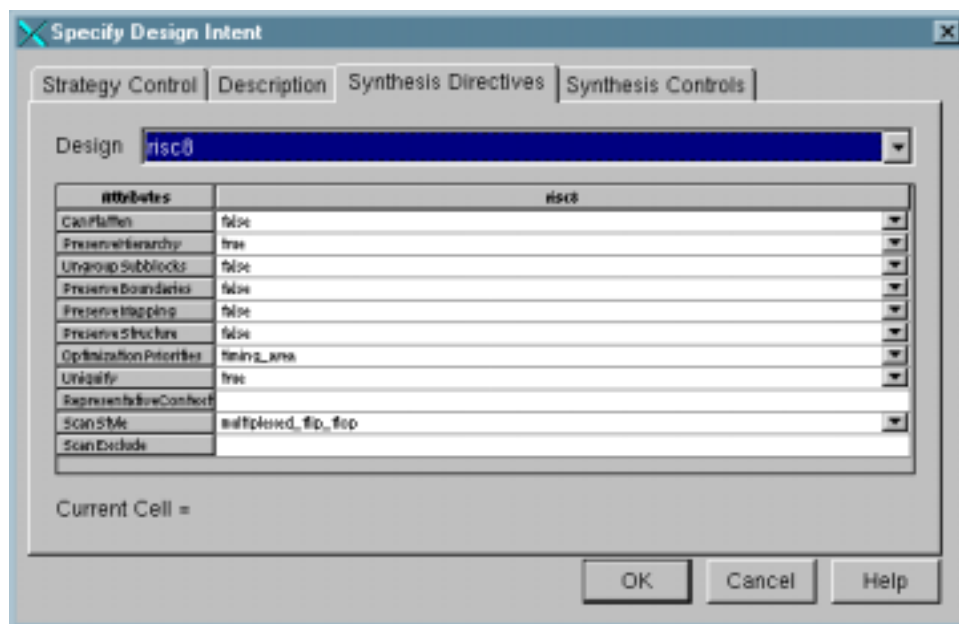
Attribute	Value
PartNameLoad	
Util LoadCode	
Util LoadCode	AreaBasedMinUsedSelection
Util LoadCode	
Util LoadCode	
OperatingConditionsWorst	slow_125C_1.62_MCT
OperatingConditionsWorst	

Current Cell =

Buttons: OK, Cancel, Help

When the user is setting the "OperatingConditionsWorst" attribute, the values that are available are the values that are read in from the technology library. The figure below shows the *Chip Context* tab of the *Specify Context Information* Activity.

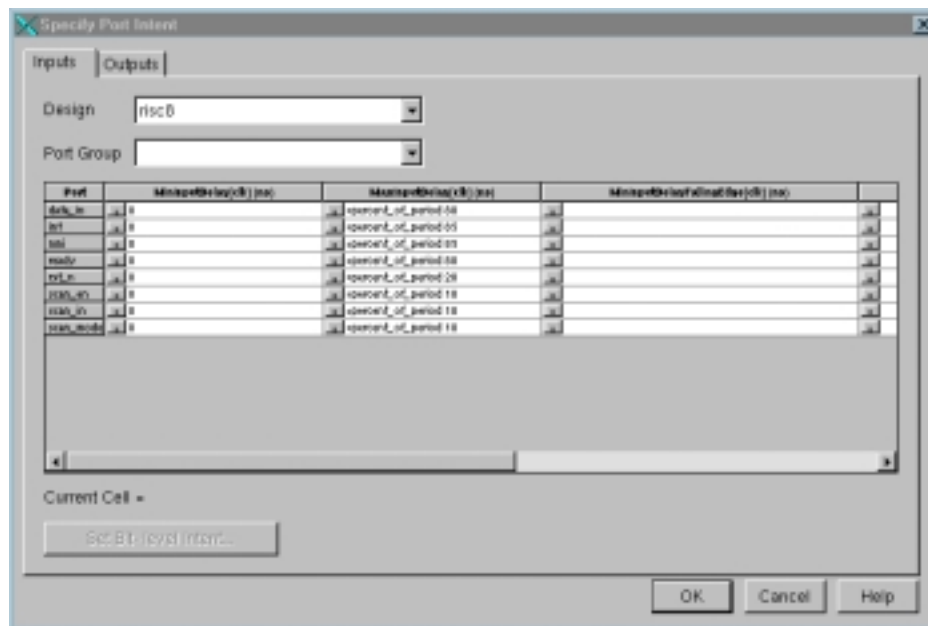
The next Activity that the user needs to update with context information is in the *Specify Design Intent* Activity, which is shown below.



As shown in the *Synthesis Directives* tab above, the user should set the "Optimization Priorities" attribute for the compile strategy that best matches the requirements of the SoC. The available settings include: timing, timing_area, area_timing, and area. The user will find that most cores are shipped with the timing_area strategy, which is the default and will give the best results where timing is important, but the user may also want the area to be reduced by Design Compiler.

Our last activity for setting the context information is the *Specify Port Intent* activity. In this activity, the user will be presented a list of all the ports, where the user can update the

timing and loading constraints to match the SoC requirements. Keep in mind, that the core developer has set these constraints based on their knowledge of the core and the best way that it should be used. In the figure below, the user can see that there are 8 inputs listed for me to update.



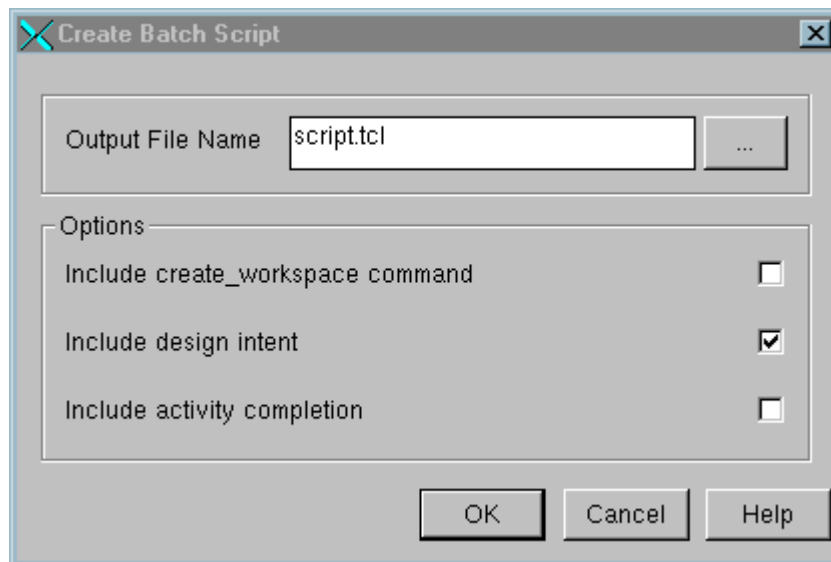
The core developer should have used the built in formulas to ensure technology portability. For example, the user can see the MaxInputDelay is relative to the clock, so when we changed the clock from 10ns to 8ns earlier, this timing constraint will scale automatically. Similarly, there are formulas for specifying the loading on the ports.

As the user of the core, the constraints can be updated at the users discretion. For our example, the group of 8 inputs would not take us long to update, especially with the copy/paste features that are built into the spreadsheet. However, for cores that have a large number of ports with different timing and loading constraints, the user may want to specify the values in a batch file and read them into coreConsultant. Please see the command line section for how to do this.

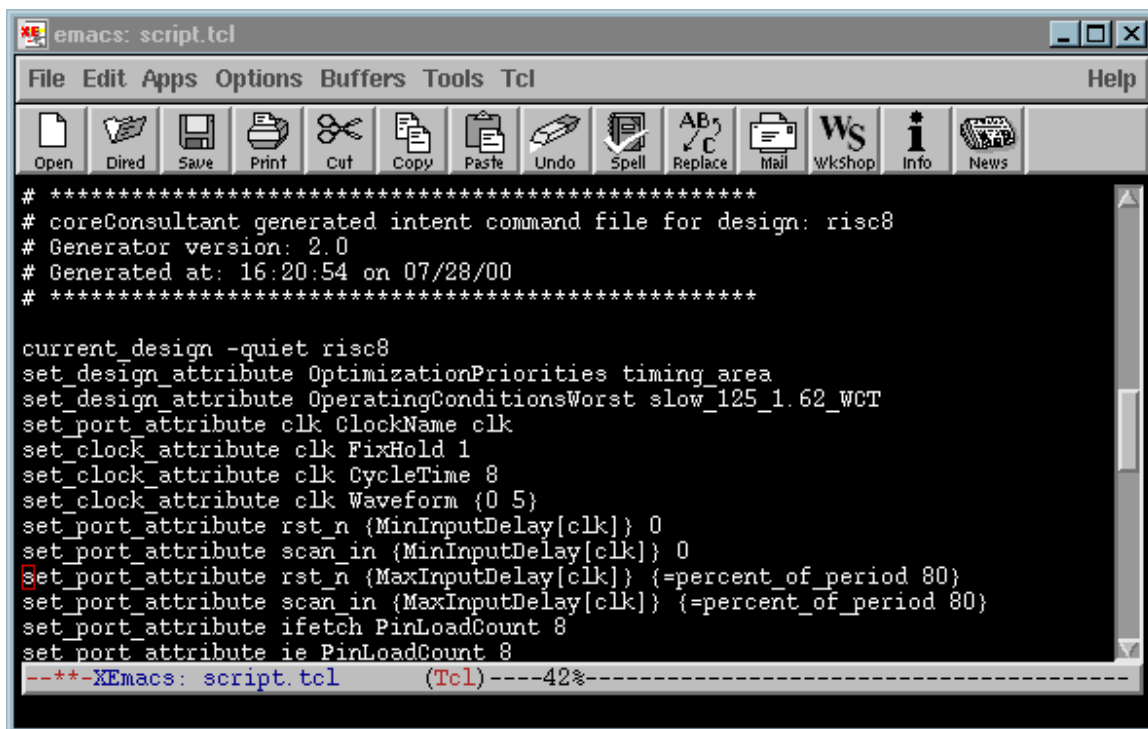
Updating Core Context Using a File

Updating the context constraints via the coreConsultant GUI is a simple task, although for a large number of input and output ports, this can take a long time. In addition, the user may want to write out all of the values that the user entered, to be able to read them into another configuration later or for archival purposes.

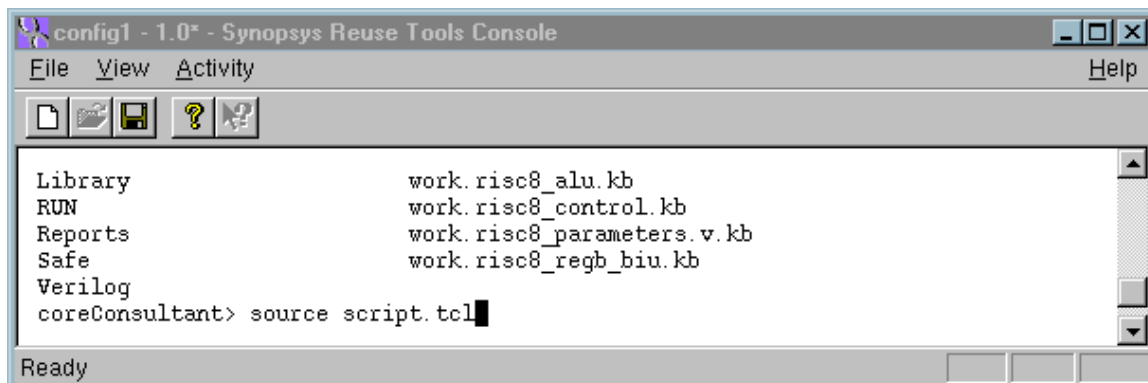
To update the context information from a file without using the GUI, the user should use the `write_batch_script` command. This command can be accessed from File>Write Script or by typing the command into the console window. The command can also be entered at the **coreConsultant>** prompt when using the shell. When using the GUI to write the batch script, the user will see the following dialog:



The user should only set the “Include design intent” option, if the desire is to only get the constraints in the file. After pushing the “OK” button, the user will generate a script, which can be edited and read back in to coreConsultant. Upon examining the `script.tcl` file, the user will find all of the attributes that have been set on the core. In the figure below, the user can see that the clock “CycleTime” has been set to 8ns.



As a user of the core, the value could be modified and then read back into coreConsultant. As an example, if the clock "CycleTime" is set to 12ns and the MaxInputDelay is set to 90% of the clock, the new values can be loaded by using the Tcl source command in the console window as shown below or in a batch file.



After sourcing the file, the user will see that the new constraints have been set if they check the values.

The result is that the user can use this mechanism to update the existing constraints, document their existing workspace, or to create a batch mechanism to update their constraints.